Building Your First AI Chip: Hackathon Starter Guide

Hackathon Mentor

Starter Project Template

- You can find all the resources, including this presentation, at: **https://github.com/stateless-loop-58/ai-chip-starter**

- This repo provides a starter Verilog implementation of a 4x4 matrix multiplier, alongside example of Python-based neural network and CNN models that run on it. Tested on a Xilinx Kria KV260 FPGA board.

- This project template is *optional* for the AI chip hackathon. If you already have your own hardware architecture, feel free to ignore this template and use your custom design instead.

What is a Neural Processing Unit (NPU)?

Specialized silicon optimized for vector and matrix math, drastically accelerating AI workloads over general-purpose CPUs and GPUs.

Cloud & Data Center

Hundreds of Watts to Kilowatts

Massive compute clusters for training and serving LLMs and foundation models.

Examples: Google TPU, AWS Trainium, NVIDIA Tensor Cores

Automotive & Edge Server

Tens to ~100 Watts

High-throughput, real-time vision processing for autonomous systems.

Examples: Tesla FSD Chip, NVIDIA Drive Thor

Mobile & Client PC

Single Digit Watts

On-device generative AI, computational photography, and camera audio effects.

Examples: Apple Neural Engine, Qualcomm Hexagon

TinyML / Micro-NPU

Milliwatts to Microwatts

Ultra-low-power microcontrollers running continuous keyword and sensor inference.

Examples: Arm Ethos-U, GreenWaves GAP9

Key Takeaway: NPUs span from supercomputers to battery-powered sensors; TinyML targets the extreme low-power microcontroller domain.

NPU Architecture Methods

INT Quantization

Hardware Matrix Multiplication

Model Optimization

Model Pruning

Hardware Tiling

Hardware Dataflow

Hardware Activation

Double Buffering

Weight Compression

Custom Instructions

INT Quantization

Floating-Point 32 (FP32)

Single-precision numeric format providing high dynamic range, standard for training deep neural networks.

32-Bit Structure

1 sign bit, 8 exponent bits, and 23 fraction bits representing numbers across massive scales.

High Precision

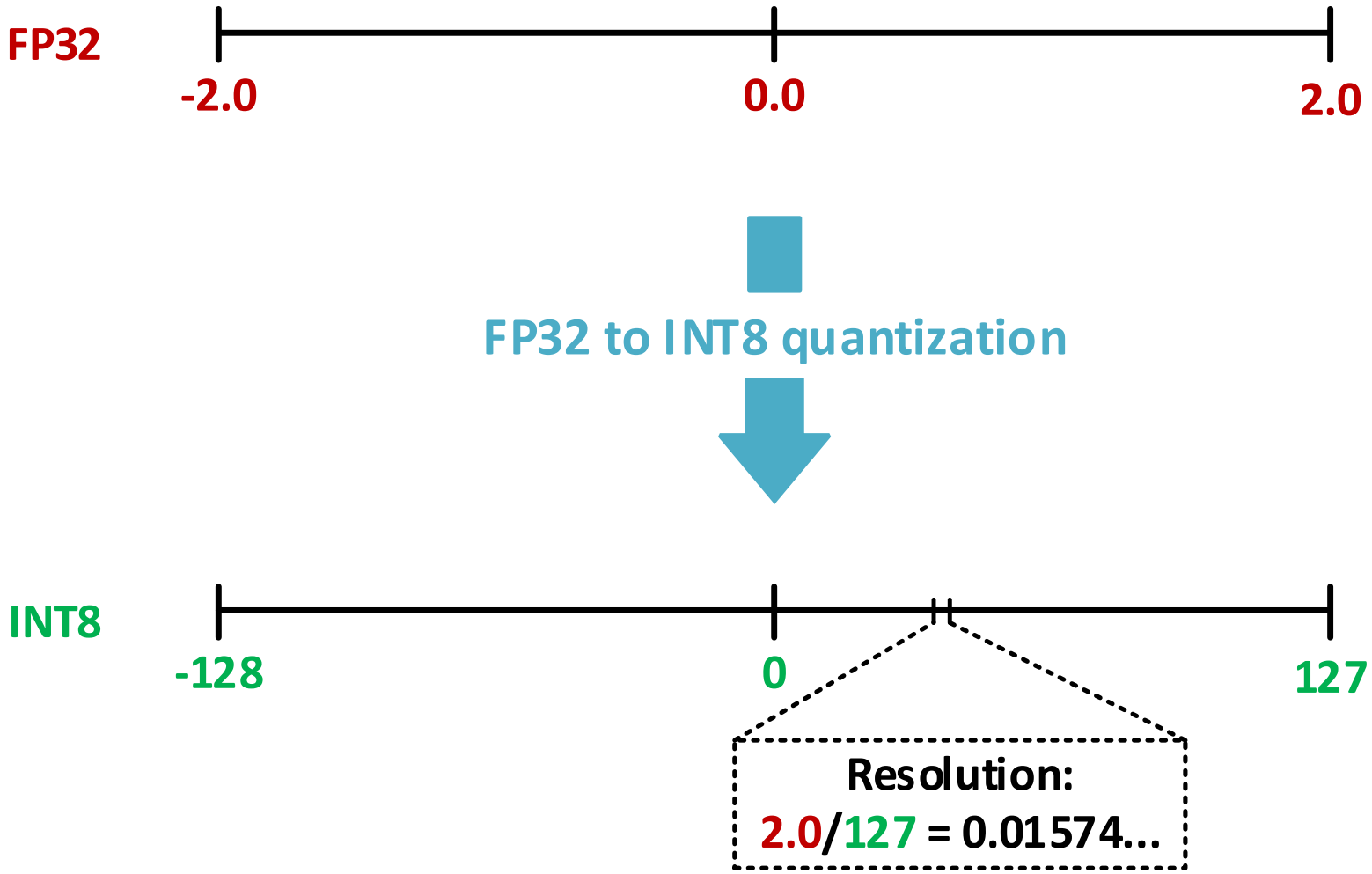
Prevents underflow and numerical instability during gradient descent weight updates.

Hardware Penalty

Demands complex floating-point logic units, higher memory bandwidth, and heavy power draw.

Key Takeaway: While vital for training, FP32 is heavily constrained on edge hardware, driving the need for INT8 quantization.

FP32 to INT8 Quantization Example



Quantization Formula

1. Calculate the quantization scale

Given an FP32 tensor x:

$$S = \frac{\max(|x|)}{127}$$

where:

- S = quantization scale
- $max(|x|)$ = maximum absolute FP32 value
- 127 = maximum positive INT8 value

2. Quantize FP32 → INT8

To quantize FP32 value to INT8:

$$x_{int8} = \text{clip}(\text{round}(\frac{x_{fp32}}{S}), -128, 127)$$

3. Dequantize INT8 → FP32

To approximately recover the original FP32 value:

$$x_{fp32} \approx x_{int8} \times S$$

Example

Suppose the FP32 tensor has values between:

$$-2.0 \text{ and } +2.0$$

Then:

$$S = 2.0 \ / \ 127 \approx 0.01575$$

For an FP32 value:

$$x = 1.0$$

Quantization:

$$x_{int8} = round(1.0 \ / \ 0.01575) \approx 64$$

Dequantization:

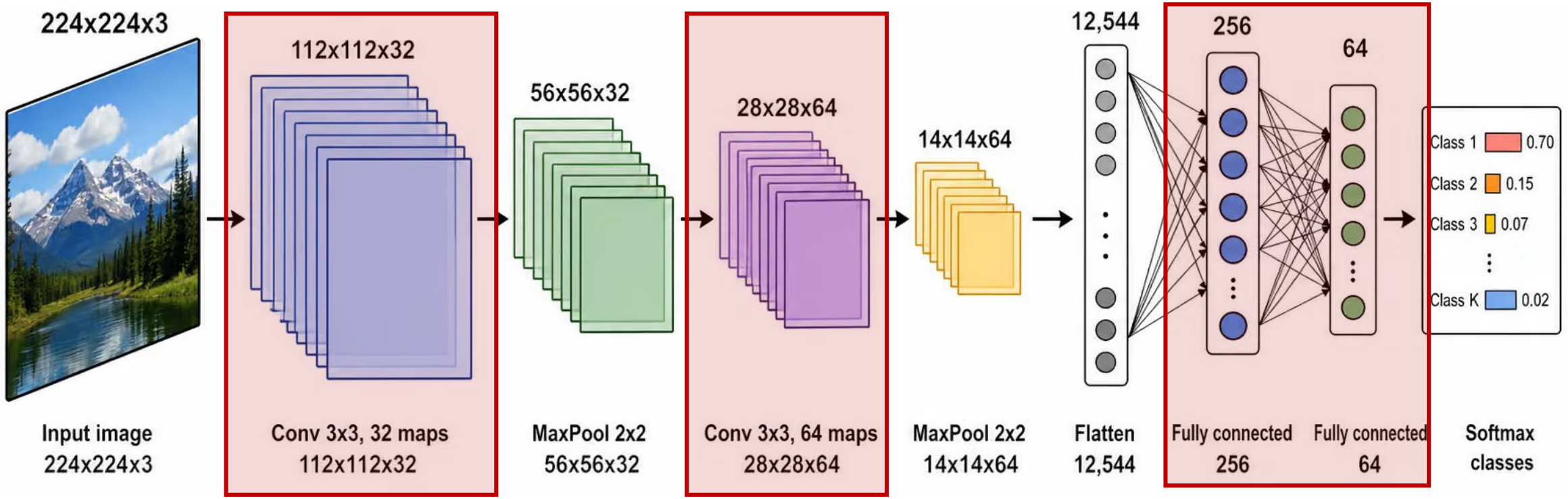
$$x_{fp32} \approx 64 \times 0.01575 \approx 1.008$$

There is a small quantization error:

$$error = \ |1.0 - 1.008| \ \approx 0.008$$

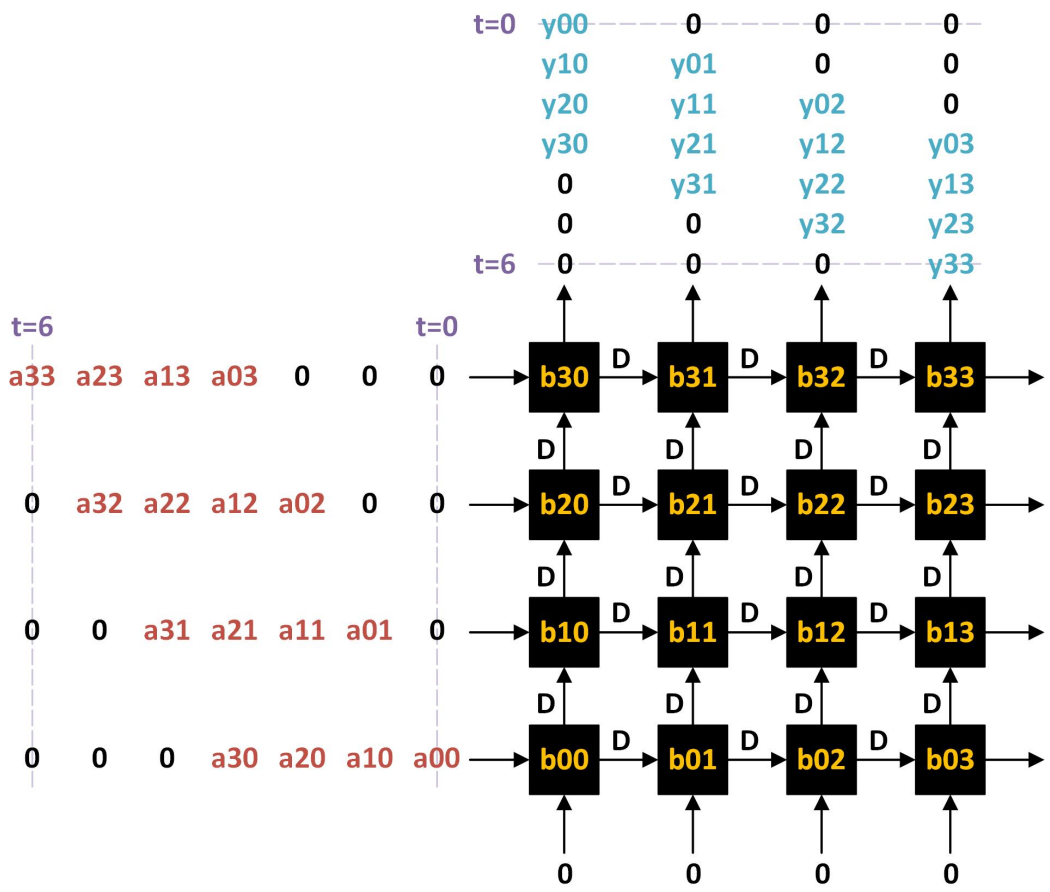
Hardware Matrix Multiplication

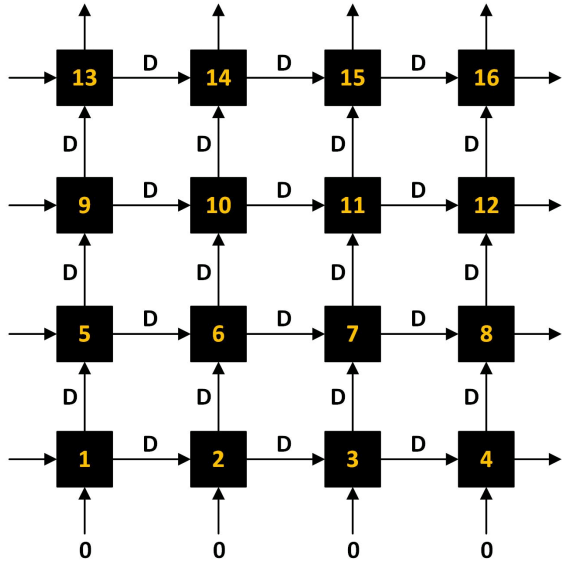
Deep Learning Architecture Example



Key Takeaway: Deep learning architectures rely on key layers like **Linear** and **Convolutional**, both of which can be computed using **matrix multiplication**.

Matrix Multiplication in Hardware



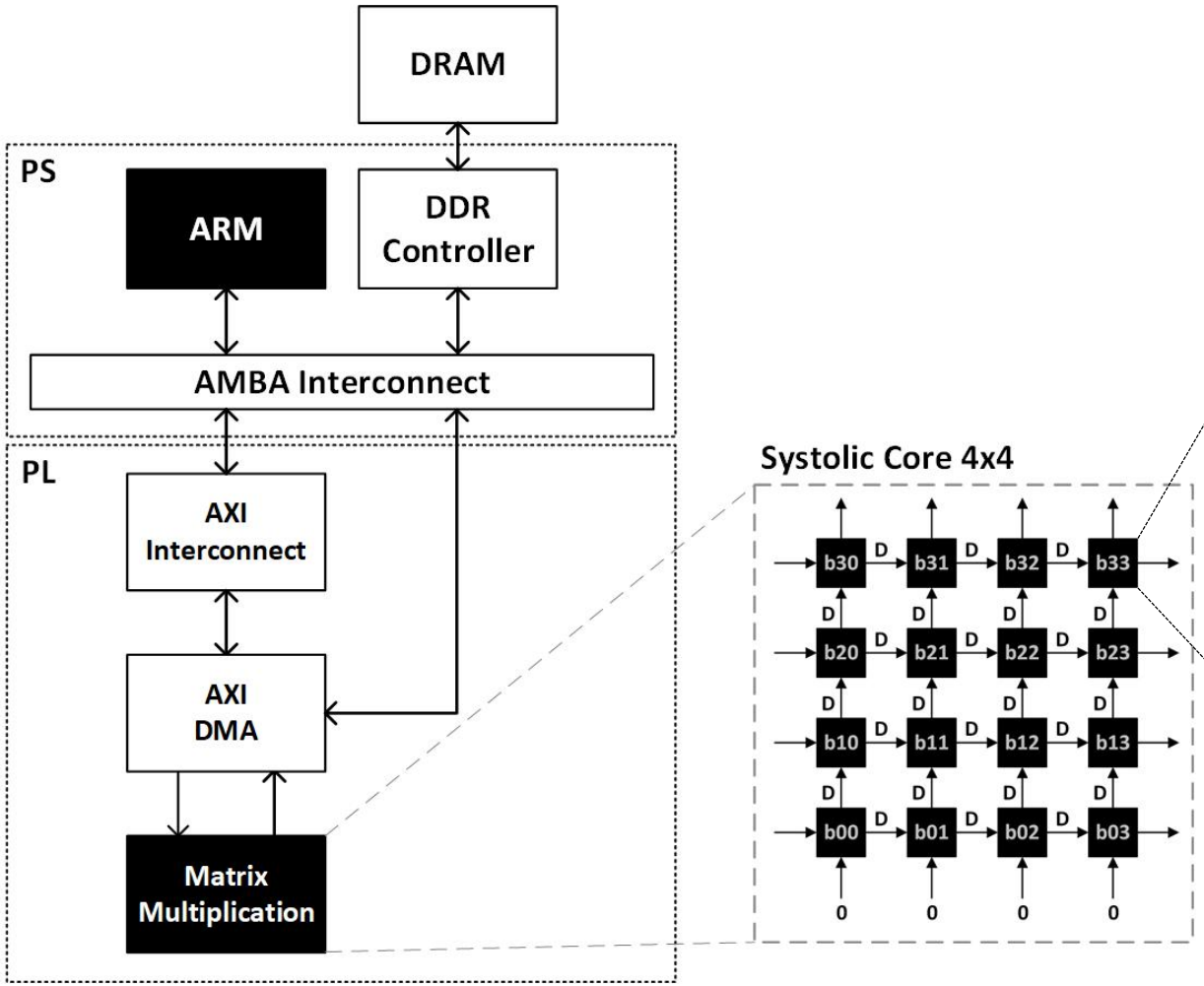


$$\mathbf{Y} = \mathbf{A} \times \mathbf{B} =$$

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix} \times \begin{bmatrix} b_{00} & b_{01} & b_{02} & b_{03} \\ b_{10} & b_{11} & b_{12} & b_{13} \\ b_{20} & b_{21} & b_{22} & b_{23} \\ b_{30} & b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} y_{00} & y_{01} & y_{02} & y_{03} \\ y_{10} & y_{11} & y_{12} & y_{13} \\ y_{20} & y_{21} & y_{22} & y_{23} \\ y_{30} & y_{31} & y_{32} & y_{33} \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} = \begin{bmatrix} 90 & 100 & 110 & 120 \\ 202 & 228 & 254 & 280 \\ 314 & 356 & 398 & 440 \\ 426 & 484 & 542 & 600 \end{bmatrix}$$

Matrix Multiplication Integrated Into SoC




```
1   module pe
2       (
3           input  wire signed [7:0]  a_in,
4           input  wire signed [31:0] y_in,
5           input  wire signed [7:0]  b,
6           output wire signed [7:0]  a_out,
7           output wire signed [31:0] y_out
8       );
9
10      wire signed [31:0] mult;
11
12      assign a_out = a_in;
13      assign mult  = a_in * b;
14      assign y_out = y_in + mult;
15
16  endmodule
```

Python Function Call to HW Matrix Multiplier

In [4]:
```
A = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12],
    [13, 14, 15, 16]], dtype=np.int64)

B = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12],
    [13, 14, 15, 16]], dtype=np.int64)
```

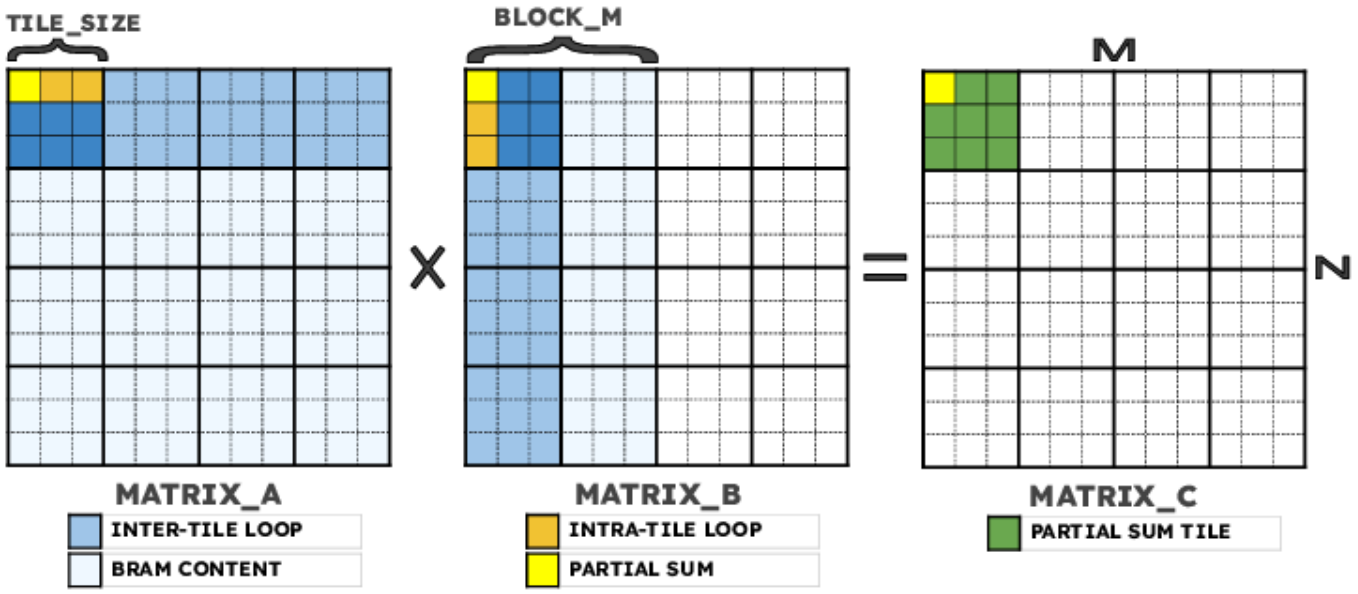
In [5]:
```
C = systolic_mmult(A, B)
C
```

Out[5]:
```
array([[ 90, 100, 110, 120],
       [202, 228, 254, 280],
       [314, 356, 398, 440],
       [426, 484, 542, 600]], dtype=int32)
```

In [6]:
```
D = np.matmul(A, B)
D
```

Out[6]:
```
array([[ 90, 100, 110, 120],
       [202, 228, 254, 280],
       [314, 356, 398, 440],
       [426, 484, 542, 600]])
```

Block Matrix Multiplication (Tiling)




```python
def block_matmul(a, b, block_size=4):
    # Get the dimensions of the input matrices
    m, n_a = a.shape
    n_b, p = b.shape
    if n_a != n_b:
        raise ValueError("Matrix dimensions must be compatible for multiplication")

    # Padding matrix a
    num_row_pad_a = int(np.ceil(m / block_size) * block_size - m)
    num_col_pad_a = int(np.ceil(n_a / block_size) * block_size - n_a)
    padded_row_a = np.vstack([a, np.zeros((num_row_pad_a, a.shape[1]))])
    padded_a = np.hstack([padded_row_a, np.zeros((padded_row_a.shape[0], num_col_pad_a))])

    # Padding matrix b
    num_row_pad_b = int(np.ceil(n_b / block_size) * block_size - n_b)
    num_col_pad_b = int(np.ceil(p / block_size) * block_size - p)
    padded_row_b = np.vstack([b, np.zeros((num_row_pad_b, b.shape[1]))])
    padded_b = np.hstack([padded_row_b, np.zeros((padded_row_b.shape[0], num_col_pad_b))])

    # Result matrix c with padding
    c = np.zeros((padded_a.shape[0], padded_b.shape[1]))

    # Iterate over blocks (using padded matrices!)
    for i in range(0, padded_a.shape[0], block_size):
        for j in range(0, padded_b.shape[1], block_size):
            for k in range(0, padded_a.shape[1], block_size):
                A_block = padded_a[i:i+block_size, k:k+block_size]
                B_block = padded_b[k:k+block_size, j:j+block_size]

                # Multiply the blocks
                C_block = systolic_mmult(A_block.astype(np.int8), B_block.astype(np.int8))

                # Accumulate the result
                c[i:i+block_size, j:j+block_size] += C_block

    # Remove padding
    return c[:m, :p]
```

Call 4x4 matrix multiplier

Example 1: Neural Network

Example 1: Neural Network

Neural network example for food preference classification.

Name	Sour	Sweet	Salty	Spicy	Label
Nasi Tim Ayam	1	6	5	1	1 (Like)
Telur Balado	2	4	6	7	0 (Dislike)
Sayur Bening Bayam	2	3	4	1	0 (Dislike)
Ayam Goreng Mentega	2	8	6	2	1 (Like)
Tumis Buncis Wortel	1	4	4	2	0 (Dislike)
Opor Ayam	1	5	5	2	1 (Like)

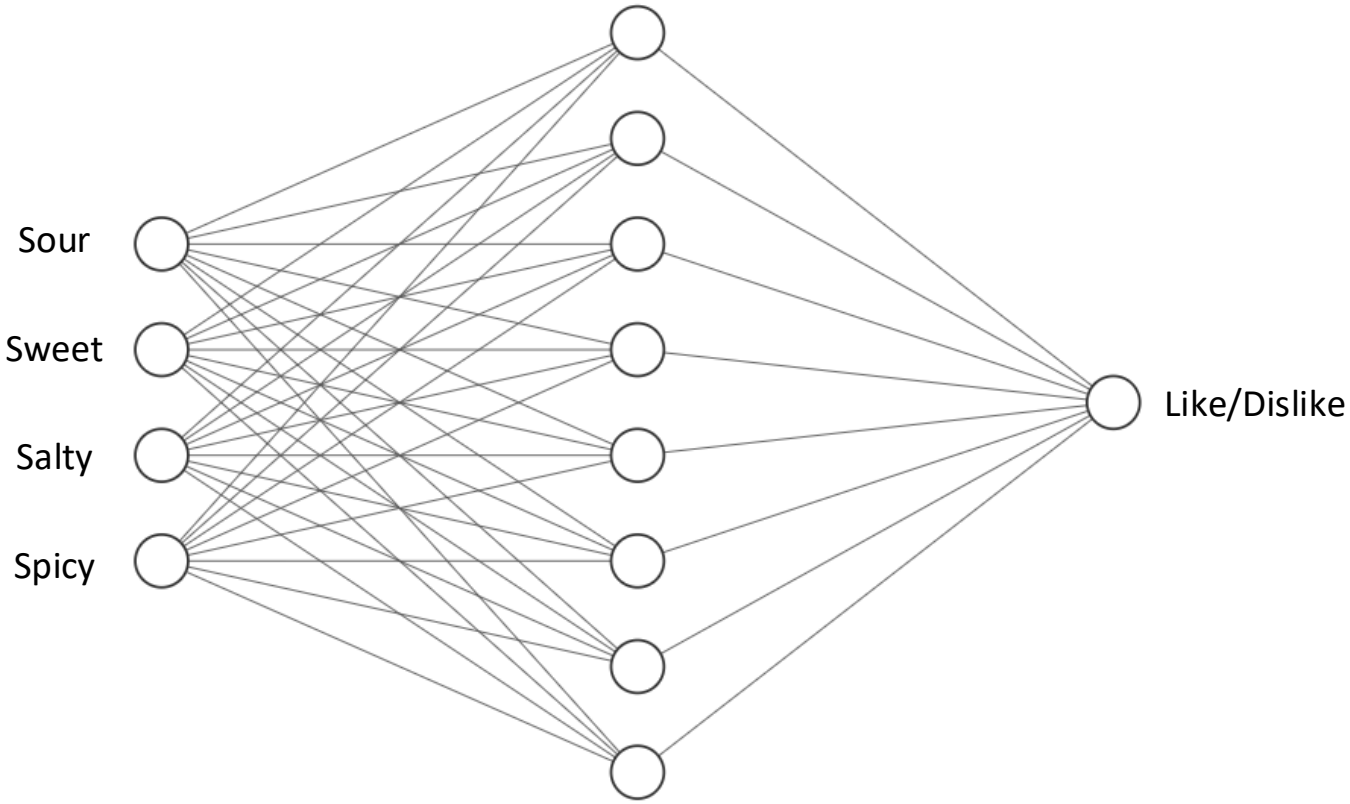






Example 1: Neural Network

Neural network architecture for food preference classification.




```python
class FoodPreferenceANN(nn.Module):
    def __init__(self, input_dim=4, hidden_dim=8):
        super(FoodPreferenceANN, self).__init__()
        self.layer1 = nn.Linear(input_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        out = self.layer1(x)
        out = self.relu(out)
        out = self.layer2(out)
        return out
```

Example 1: Neural Network

This **Python class** replaces PyTorch's default **nn.Linear** layer by integrating **INT8 quantization** and FPGA **hardware matrix multiplication**.

```python
class FPGAQuantizedLinear(nn.Module):

    def __init__(self, original_layer, input_scale, weight_scale):
        super().__init__()

        self.in_features = original_layer.in_features
        self.out_features = original_layer.out_features

        # Quantization scales
        self.input_scale = input_scale
        self.weight_scale = weight_scale

        # ------------------------------------------------------------------
        # Quantize weights using the calibrated weight scale
        # ------------------------------------------------------------------
        weights_float = original_layer.weight.detach().cpu().numpy()

        self.weights_int8 = np.round(
            weights_float / self.weight_scale
        ).clip(-128, 127).astype(np.int8)

        # Keep bias in floating point for simplicity
        if original_layer.bias is not None:
            self.bias = original_layer.bias.detach().clone()
        else:
            self.bias = None
```

Weight Quantization

```python
def forward(self, x):

    # FP32 -> INT8
    x_np = x.detach().cpu().numpy()

    x_int8 = np.round(
        x_np / self.input_scale
    ).clip(-128, 127).astype(np.int8)

    # INT8 x INT8 -> INT32
    x1_shape = x_int8.shape
    x2_shape = self.weights_int8.T.shape
    print(f"DEBUG: FPGA matmul X1{x1_shape} * X2{x2_shape}")

    # Matrix multiplication with FPGA
    out_int32 = block_matmul(
        x_int8.astype(np.int32),
        self.weights_int8.T.astype(np.int32)
    )

    # INT32 -> FP32
    total_scale = self.input_scale * self.weight_scale

    out_float = (
        out_int32.astype(np.float32) * total_scale
    )

    # Add FP32 bias
    if self.bias is not None:
        out_float += self.bias.cpu().numpy()

    return torch.from_numpy(out_float).to(x.device)
```

Input Quantization

HW Matrix Multiplier FPGA

Output Dequantization

Example 1: Neural Network

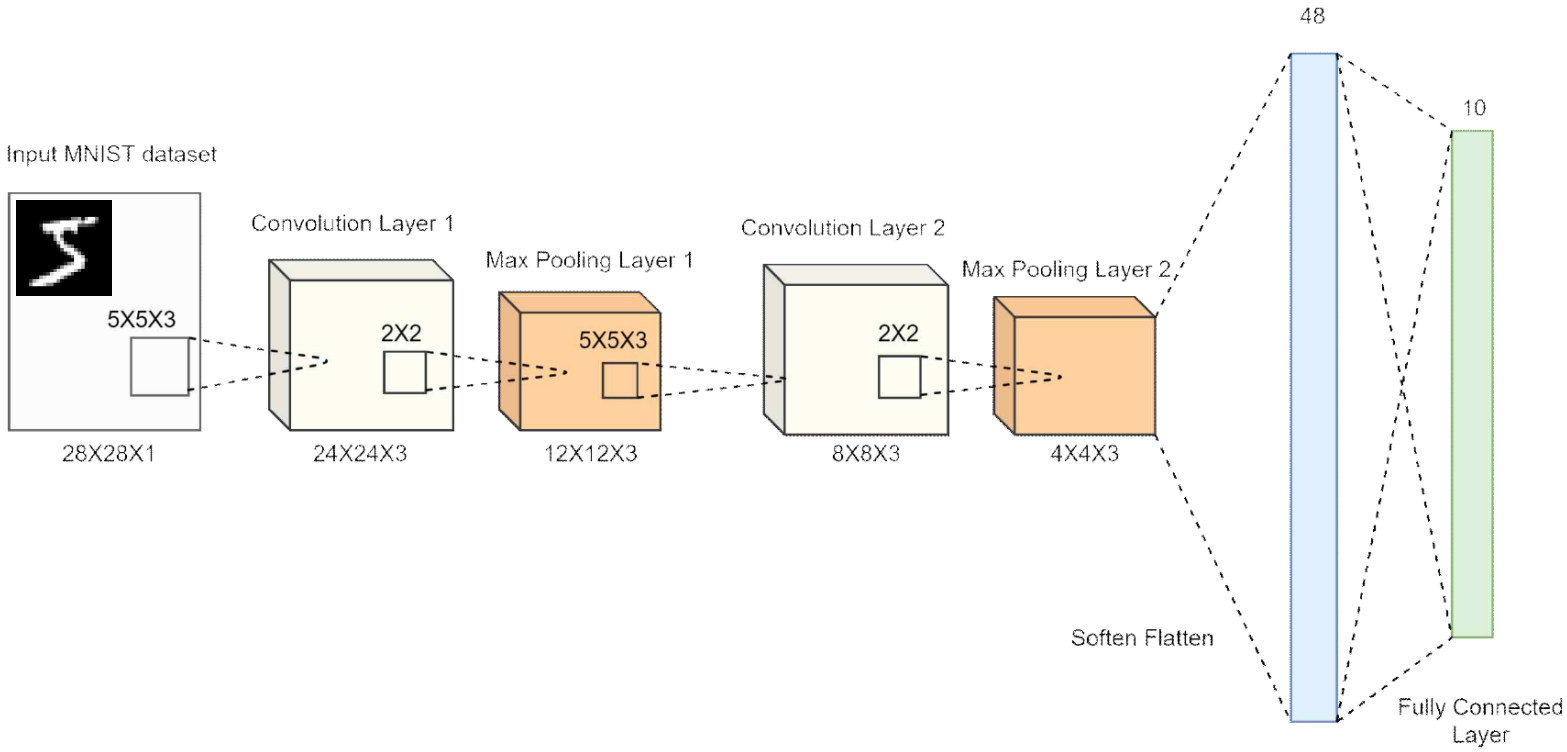
A comparison between FP32 and INT8 results, along with an evaluation of the quantization error.

```
--- FP32 vs FPGA INT8 Model ---
Food                 |  FP32 Prob |  INT8 Prob |  Abs Error |  FP32 Pred |  INT8 Pred
-------------------------------------------------------------------------------------
Nasi Tim Ayam        |     0.9892 |     0.9891 |     0.0001 |       Like |       Like
Telur Balado         |     0.0067 |     0.0067 |     0.0000 |    Dislike |    Dislike
Sayur Bening Bayam   |     0.0003 |     0.0003 |     0.0000 |    Dislike |    Dislike
Ayam Goreng Mentega  |     0.9961 |     0.9960 |     0.0000 |       Like |       Like
Tumis Buncis Wortel  |     0.0387 |     0.0382 |     0.0005 |    Dislike |    Dislike
Opor Ayam            |     0.9408 |     0.9406 |     0.0002 |       Like |       Like
-------------------------------------------------------------------------------------
Mean Absolute Error:    0.000144
Maximum Absolute Error: 0.000493
```

Example 2: Convolutional Neural Network

Example 2: Convolutional Neural Network

Convolutional neural network architecture for handwritten digit classification.




```python
def forward(self, x):
    in_size = x.size(0)

    # Layer Integration
    x = self.conv1(x)
    self.conv1_out_np = x.detach().numpy()

    x = F.relu(self.mp(x))
    self.mp1_out_np = x.detach().numpy()

    x = self.conv2(x)
    self.conv2_out_np = x.detach().numpy()

    x = F.relu(self.mp(x))
    self.mp2_out_np = x.detach().numpy()

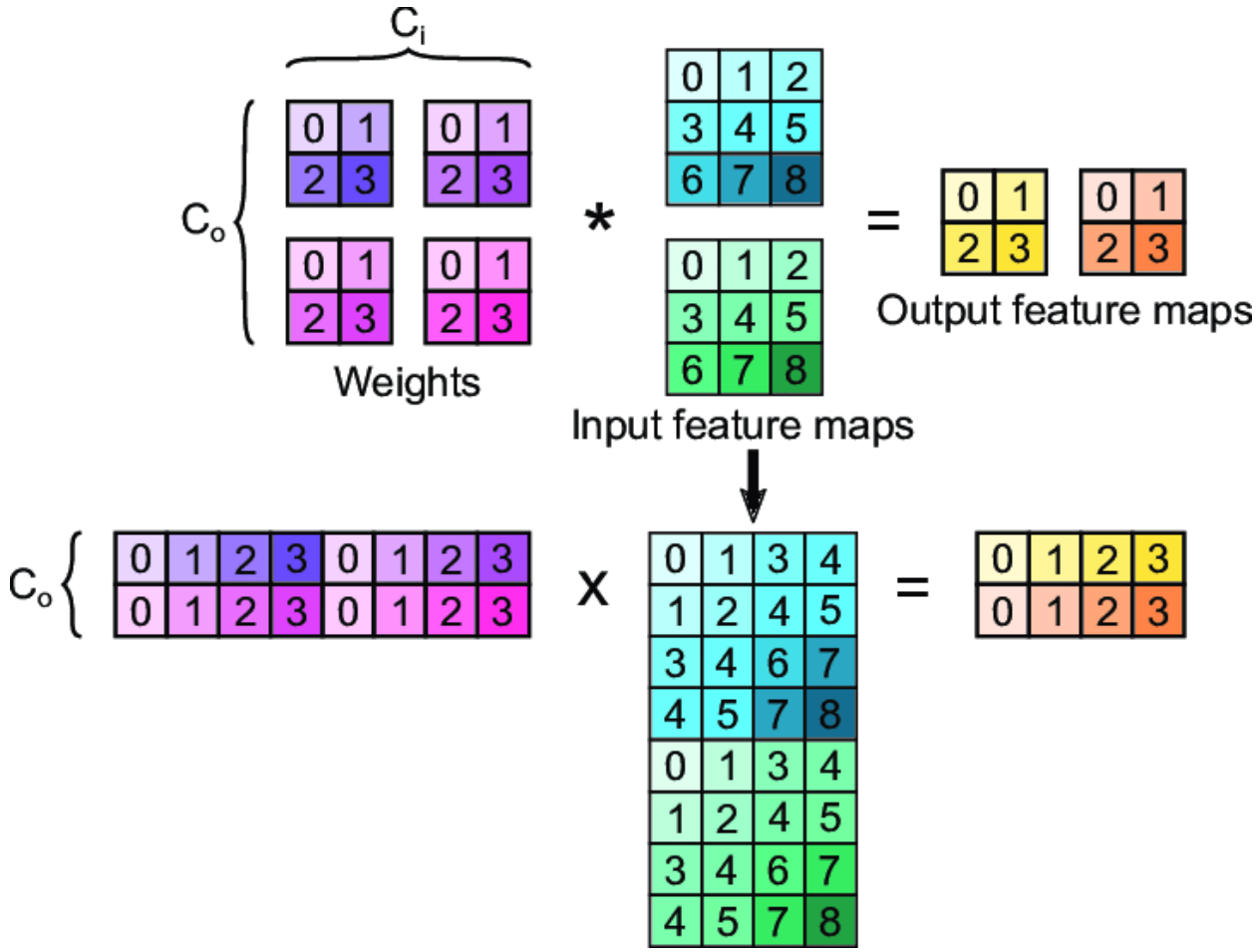
    # Flatten Layer
    x = x.view(in_size, -1)
    self.fc_in_np = x.detach().numpy()

    # Fully Connected Layer
    x = self.fc_1(x)
    self.fc_out_np = x.detach().numpy()

    return F.log_softmax(x)
```

Example 2: Convolutional Neural Network

Image-to-Column (**im2col**) converts **image feature maps** into **matrix representations** so they can be processed using standard matrix multiplication.



Example 2: Convolutional Neural Network

This **Python class** replaces PyTorch's default **nn.Conv2d** layer by integrating **INT8 quantization, im2col,** and FPGA **hardware matrix multiplication**.

```python
def forward(self, x):
    batch_size = x.size(0)
    in_h, in_w = x.size(2), x.size(3)

    # 1. FP32 -> INT8 Activation Quantization
    x_np = x.detach().cpu().numpy()
    x_int8 = np.round(
        x_np / self.input_scale
    ).clip(-128, 127).astype(np.int8)

    # Re-wrap as torch tensor to perform im2col unrolling via F.unfold
    x_int8_tensor = torch.from_numpy(x_int8).to(torch.float32)

    # Unroll spatial image patches into columns (im2col)
    # Output shape: (batch_size, in_channels * K_H * K_W, L)
    # where L = output_height * output_width
    x_unfolded = F.unfold(
        x_int8_tensor,
        kernel_size=self.kernel_size,
        dilation=self.dilation,
        padding=self.padding,
        stride=self.stride
    ).numpy().astype(np.int8)

    # Calculate spatial output dimensions
    k_h = self.kernel_size[0] if isinstance(self.kernel_size, tuple) else self.kernel_size
    k_w = self.kernel_size[1] if isinstance(self.kernel_size, tuple) else self.kernel_size
    p_h = self.padding[0] if isinstance(self.padding, tuple) else self.padding
    p_w = self.padding[1] if isinstance(self.padding, tuple) else self.padding
    s_h = self.stride[0] if isinstance(self.stride, tuple) else self.stride
    s_w = self.stride[1] if isinstance(self.stride, tuple) else self.stride

    out_h = int((in_h + 2 * p_h - k_h) / s_h + 1)
    out_w = int((in_w + 2 * p_w - k_w) / s_w + 1)

    outputs = []
```

Input Quantization

Im2col

```python
    # Process each image in the batch
    for b in range(batch_size):
        # Transpose unfolded patch matrix for matmul: (L, in_channels * K_H * K_W)
        patch_matrix = x_unfolded[b].T

        x1_shape = patch_matrix.shape
        x2_shape = self.weights_int8.T.shape
        print(f"DEBUG: matmul X1{x1_shape} * X2{x2_shape}")

        # 2. INT8 x INT8 -> INT32 Matrix Multiplication FPGA
        # (L, K_dim) * (K_dim, out_channels) -> (L, out_channels)
        out_int32 = block_matmul(
            patch_matrix.astype(np.int32),
            self.weights_int8.T.astype(np.int32)
        )

        # 3. INT32 -> FP32 Dequantization
        total_scale = self.input_scale * self.weight_scale
        out_float = out_int32.astype(np.float32) * total_scale

        # Add FP32 bias if present
        if self.bias is not None:
            out_float += self.bias  # Broadcast across channels

        # Reshape back from (L, out_channels) -> (out_channels, out_h, out_w)
        out_float = out_float.T.reshape(self.out_channels, out_h, out_w)
        outputs.append(out_float)

    # Stack batch dimension: (batch_size, out_channels, out_h, out_w)
    final_out = np.stack(outputs, axis=0)

    return torch.from_numpy(final_out).to(x.device).contiguous()
```

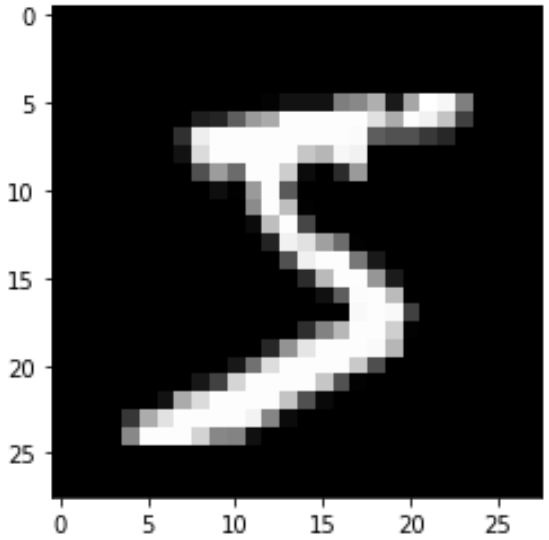
HW Matrix Multiplier FPGA

Output Dequantization

Example 2: Convolutional Neural Network

A comparison between FP32 and INT8 results, along with an evaluation of the quantization error.

```
--- FP32 vs INT8 Model Comparison ---
Sample / Class         |  FP32 Prob |  INT8 Prob |  Abs Error |  FP32 Pred |  INT8 Pred
-------------------------------------------------------------------------------------
Digit Class 0          |     0.0000 |     0.0000 |     0.0000 |            |
Digit Class 1          |     0.0000 |     0.0000 |     0.0000 |            |
Digit Class 2          |     0.0000 |     0.0000 |     0.0000 |            |
Digit Class 3          |     0.2061 |     0.2021 |     0.0040 |            |
Digit Class 4          |     0.0000 |     0.0000 |     0.0000 |            |
Digit Class 5          |     0.7935 |     0.7975 |     0.0040 |    Digit 5 |    Digit 5
Digit Class 6          |     0.0000 |     0.0000 |     0.0000 |            |
Digit Class 7          |     0.0000 |     0.0000 |     0.0000 |            |
Digit Class 8          |     0.0002 |     0.0002 |     0.0000 |            |
Digit Class 9          |     0.0001 |     0.0001 |     0.0000 |            |
-------------------------------------------------------------------------------------
Mean Absolute Probability Error:    0.000802
Maximum Absolute Probability Error: 0.004008
```



Design Flow Summary

- FP32 modeling (Pytorch)
  - Train and validate your model using floating-point precision to establish a baseline accuracy.
- INT8 quantization
  - Convert the weights, biases, and activations from FP32 down to INT8 format so they fit efficiently into hardware registers and the NPU/systolic array.
- Run on FPGA
  - Deploy the quantized weights and model logic onto the Kria KV260 FPGA hardware accelerator for high-speed, low-power inference.

Thank You